

CSE 402

University of Asia Pacific

Course Name : Operating System Lab

Lab Report 1

Topic : FSFC Algorithm

Presented by,

Fahad Hasan Mahi
Reg : 23101186
Section : D1

Presented to,

Atia Rahman Orthi
Lecturer, UAP

Problem Statement :

In a multiprogramming environment the CPU must constantly decide which of the waiting processes should execute next. This experiment implements the First Come First Serve (FCFS) policy, where processes are dispatched strictly in the order they enter the ready queue, and a running process keeps the CPU until its burst finishes (non-preemptive scheduling).

The task is to write a Python program that accepts five processes, each described by a Process ID, Arrival Time and Burst Time, and then:

- Arranges the processes in increasing order of arrival time.
- Simulates the non-preemptive execution on a single CPU to obtain each process's Completion Time (CT).
- Derives the Turnaround Time (TAT = CT − AT) and Waiting Time (WT = TAT − BT).
- Computes the average turnaround time and average waiting time.
- Prints the execution timeline (Gantt chart) so the result can be verified by hand.

Source Code :

```
cpu_schudule = [(3, 3),   # P1
       (2, 1),   # P2
       (5, 2),   # P3
       (0, 3),   # P4
       (1, 2)]   # P5

def fcfs(cpu_schedule):
   procs = [(f"P{i+1}", at, bt) for i, (at, bt) in enumerate(cpu_schedule)]

   order = sorted(procs, key=lambda p: (p[1], p[0]))

   time, results, gantt = 0, [], []

   for pid, at, bt in order:

      if time < at:
         gantt.append(("IDLE", time, at))
         time = at

start = time
      ct    = start + bt       # Completion Time
      tat   = ct - at          # Turnaround Time
      wt    = tat - bt         # Waiting Time

      gantt.append((pid, start, ct))
      results.append([pid, at, bt, ct, tat, wt])
      time = ct

   return results, gantt

def print_results(results):
   print(f"{'PID':<5}{'AT':>3}{'BT':>4}{'CT':>4}{'TAT':>5}{'WT':>4}")
   print("-" * 25)
   for pid, at, bt, ct, tat, wt in sorted(results):
```

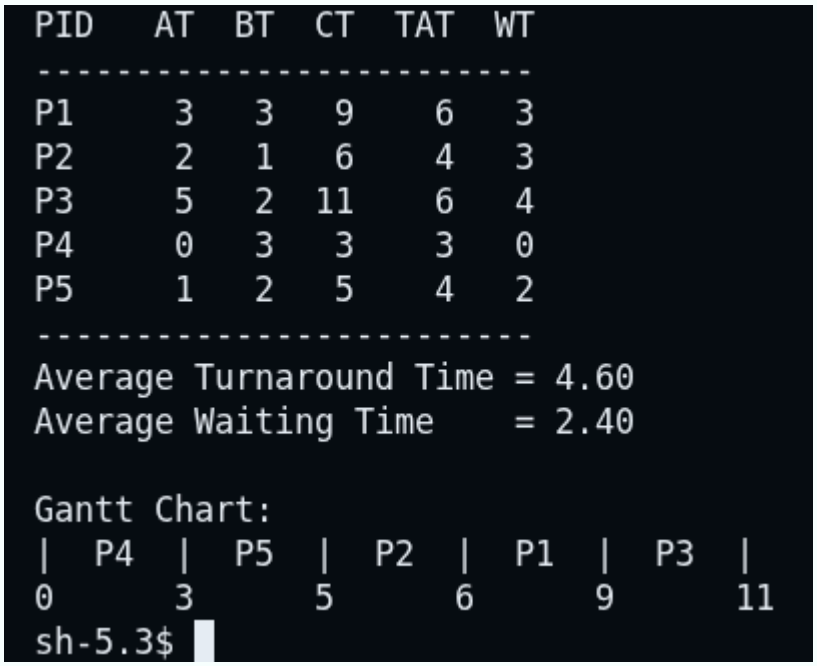
```
        print(f"{pid:<5}{at:>3}{bt:>4}{ct:>4}{tat:>5}{wt:>4}")

    n = len(results)
    print("-" * 25)
    print(f"Average Turnaround Time = {sum(r[4] for r in results) / n:.2f}")
    print(f"Average Waiting Time    = {sum(r[5] for r in results) / n:.2f}")


def print_gantt(gantt):
    print("\nGantt Chart:")
    top, bottom = "", ""
    for pid, s, e in gantt:
        top    += f"| {pid:^4} "
        bottom += f"{s:<7}"
    top    += "|"
    bottom += str(gantt[-1][2])
    print(top)
    print(bottom)


if __name__ == "__main__":
    results, gantt = fcfs(cpu_schudule)
    print_results(results)
    print_gantt(gantt)
```

Output Screenshot :



Discussion :

In this experiment the FCFS policy was simulated in Python and the output was cross-checked against hand calculations. Because P4 has the earliest arrival (AT = 0), it captures the CPU first and runs for its full burst of 3 units. After that, the CPU always picks whichever process has been waiting the longest: P5, then P2, P1 and finally P3, producing the timeline. Since a process was always available, the CPU never idled and the total makespan (11) equals the sum of all burst times.

The results confirm the standard scheduling identities. For instance, P3 finishes at 11, so its turnaround is 11 − 5 = 6, and removing its burst of 2 leaves a waiting time of 4. Across the five processes the averages come out to 4.60 (TAT) and 2.40 (WT), matching the manual calculation exactly.

The trace also exposes the classic weakness of FCFS: the algorithm looks only at arrival order and ignores burst length. P3, although it needs only 2 units, waits 4 units behind longer jobs, and P1 waits 3 units despite arriving at time 3. This "convoy" behaviour is precisely why FCFS alone is unsuitable for interactive systems, though its determinism and starvation-free nature keep it useful as a building block inside more advanced schedulers.

Advantages observed :

- Trivial to implement - a single FIFO ready queue is enough.
- Deterministic behaviour makes results easy to verify and debug.
- Fair in arrival order; every process is guaranteed CPU access, so starvation cannot occur.
- Very low scheduling overhead, since the CPU switches only when a process finishes or blocks.

Disadvantages observed :

- Non-preemptive: one long job can hold the CPU and delay everything behind it.
- Average waiting time is generally higher than SJF, SRTF or Round Robin.
- Suffers from the convoy effect, short jobs trapped behind a long one.
- Poor response time for time-sharing/interactive workloads.
- Ignores priorities and deadlines entirely.

Conclusion :

This FCFS program sorted the processes by arrival time, simulated non-preamptive execution, and produced completion, turnaround and waiting times that agreed with both the hand calculations and the Gantt chart, yielding an average turnaround time of 4.60 and an average waiting time of 2.40. The exercise confirmed that FCFS is simple, predictable and free of starvation, but also demonstrated its key limitation: because burst time is ignored, short processes may wait disproportionately long behind longer ones (the convoy effect). For this reason, real operating systems rarely rely on pure FCFS, yet its first-come-first-served principle remains the foundation of more sophisticated policies such as multilevel feedback queues.